



# Unity Catalog

Unified governance for all  
data and AI assets



# Spark – Challenges of Data Processing

- Spark has no "memory" of data structures. Every time a user wants to interact with data, they must provide the full context manually.
- You must specify the **path**, the **format**, and often the **schema** for every single query.
  - `spark.read.format("parquet").load(".....")`
- If you create a "table" in one notebook, another notebook or cluster has no idea it exists.
- Every developer must know the exact schema or rely on "schema inference," which is expensive and slow for large datasets. no idea it exists.



# Spark Catalog

- **The Metadata Registry:** The central repository for tracking databases, tables, partitions, views, and functions.
  - Manages both **temporary** (session-scoped) and **persistent** (global) metadata.
- **SessionCatalog:** The entry point. It manages the lifecycle of temporary tables and interacts with the persistent store.
- **ExternalCatalog:** The bridge to permanent storage.
  - *In-Memory:* Default, non-persistent (great for testing).
  - *Hive Metastore:* The industry standard for production persistence.

We've been using this option so far!



## Spark Native Catalog Issues

### 1. The "Single Session" Lock (Embedded Mode)

- **The Issue:** By default, Spark uses an embedded **Derby** database for the catalog. Derby only allows **one SparkSession at a time** to access metadata.
- **The Impact:** If you try to launch two notebooks or two jobs at once, the second one will fail with a "Metastore is locked" error.
- **The Fix:** In a multi-user environment, you *must* move away from default settings and use a standalone **Hive Metastore** (MySQL/Postgres).

### 2. Metadata Drift (Out-of-Sync State)

- **The Issue:** The catalog is a "pointer." If files are deleted or added directly in S3/HDFS (bypassing Spark), the catalog doesn't know.
- **The Impact:** Queries fail with `FileNotFoundException` or simply miss new data.
- **The Fix:** Manual intervention is required:
  - `REFRESH TABLE table_name` (reloads existing metadata)
  - `MSCK REPAIR TABLE table_name` (MSCK → Metastore System Check: discovers new partitions on disk).

### 3. The "Small File" Performance Trap

- **The Issue:** The catalog stores partition metadata. If you have a table partitioned by `user_id` with 1,000,000 IDs, the catalog must track 1,000,000+ entries.
- **The Impact:** "**Metastore Overload.**" Every time Spark plans a query, it asks the catalog for the file list. High-cardinality partitioning causes massive latency in the planning phase before the data is even read.
- **The Fix:** Use coarse-grained partitioning (e.g., date or region) and keep partition counts in the

low thousands, Use Delta to manage partitions, etc.

# Spark With Hive Metastore (HMS)

- When you add a metastore, you introduce a **Catalog Layer**.
  - *Spark now stores the mapping between a table name and the underlying files.*
- You define the table once; after that, you use standard SQL.
  - `spark.sql("SELECT * FROM users")`
- Multiple clusters can connect to the same HMS. If Team A updates the sales table, Team B sees the update immediately.
- Spark fetches the schema from the metastore (milliseconds) instead of scanning files in storage to infer it



<https://www.sparkcodehub.com/pyspark/sql/catalog-api>

The Catalog API in PySpark, found under `pyspark.sql.catalog.Catalog`, is a programmatic interface for interacting with Spark SQL's metadata catalog. You access it via the `catalog` attribute of a [SparkSession](#), and it serves as a control panel for everything Spark knows about its tables, databases, and functions. Think of it as a directory that tracks what's available for [spark.sql](#) queries—whether it's temporary views created with [createTempView](#), global views from [createGlobalTempView](#), or persistent tables stored in [Hive](#). It's not just a passive list; it lets you actively manage these objects—create databases, drop tables, or even tweak caching—all while Spark's [architecture](#) handles the distributed heavy lifting.

You can use it to list all tables in a database, check if a table exists, fetch its schema, or inspect registered functions, all without writing SQL queries. It's especially handy when you're working with structured data in [DataFrames](#) and need to ensure your environment is set up correctly, whether for debugging, automation, or integration with external systems like [Hive tables](#). The Catalyst optimizer ties it all together, ensuring that metadata operations align with Spark's execution engine.

# Spark Hive Setup

```
# Enable Hive support
```

```
spark.sql.catalogImplementation hive
```

```
# Warehouse Location
```

```
spark.sql.warehouse.dir <file_path>
```

```
# Database Connection (Hive Database Connection)
```

```
spark.hadoop.javax.jdo.option.ConnectionURL  
    jdbc:postgresql://postgres-db:5432/metastore
```

```
...
```

```
Other connection options
```



**Property:** spark.sql.catalogImplementation: hive

Tells Spark to use the Hive external catalog instead of the default in-memory catalog.

When a user runs `CREATE TABLE student_data (id INT)`, Spark manages both the **metadata** (in the Hive Metastore) and the **physical data**. The physical data is stored in a sub-folder inside `spark.sql.warehouse.dir`.

*Behavior:* If you `DROP TABLE`, Spark deletes **both** the metadata and the actual data files.

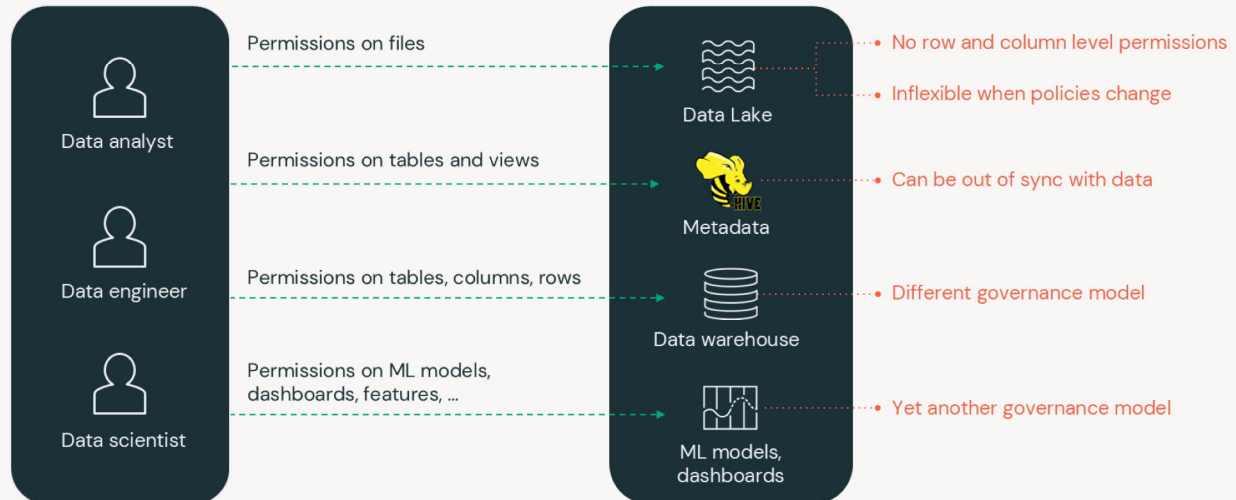
## Spark + Hive Shortcomings

- Hive Metastore (HMS) was built for **single-cluster metadata management**.
- Hive Metastore operates **per-workspace or per-cluster**, causing metadata silos and inconsistent policies.
- HMS supports only coarse, table-level permissions
- Hive Metastore does **not** capture lineage or track governance/audit events



<https://www.databricks.com/blog/enterprise-scale-governance-migrating-hive-metastore-unity-catalog>

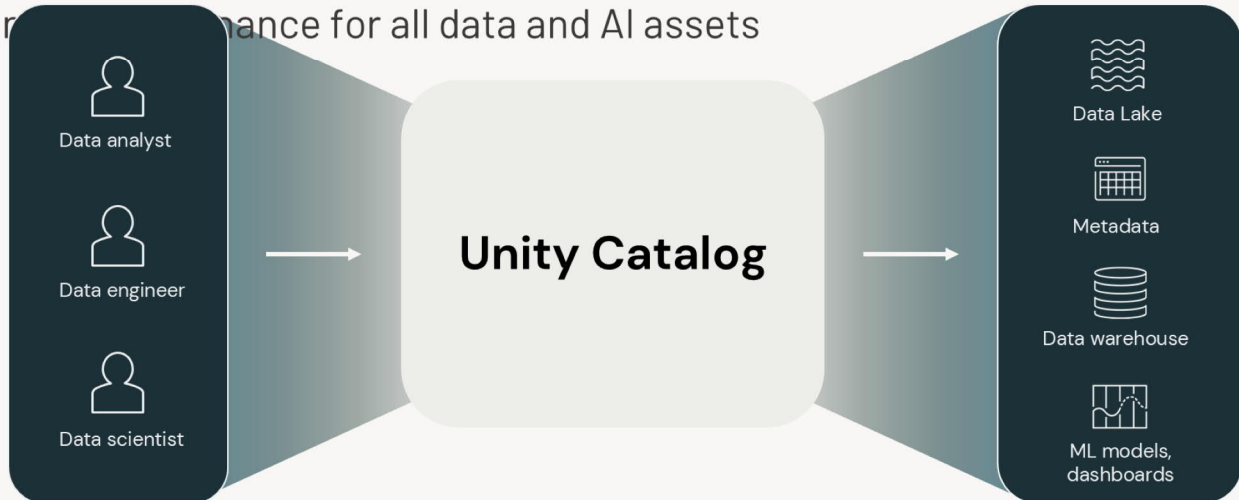
# Governance for data and AI is complex





# Databricks Unity Catalog

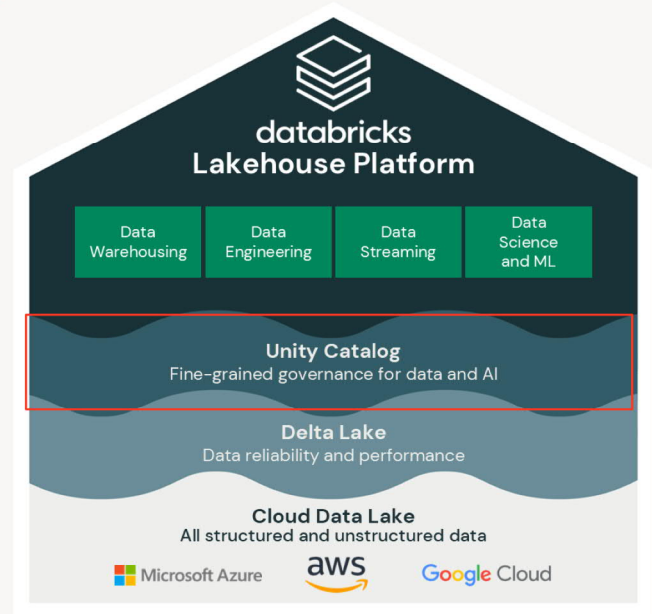
Unity Catalog provides a single source of truth and governance for all data and AI assets



# Databricks Unity Catalog

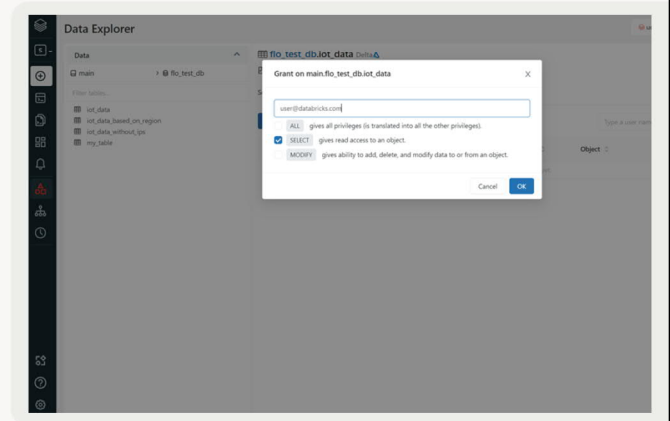
Unified governance for all data and AI assets

- Centralized governance for data and AI
- Built-in data search and discovery
- Performance and scale
- Automated lineage for all workloads
- Integrated with your existing tools



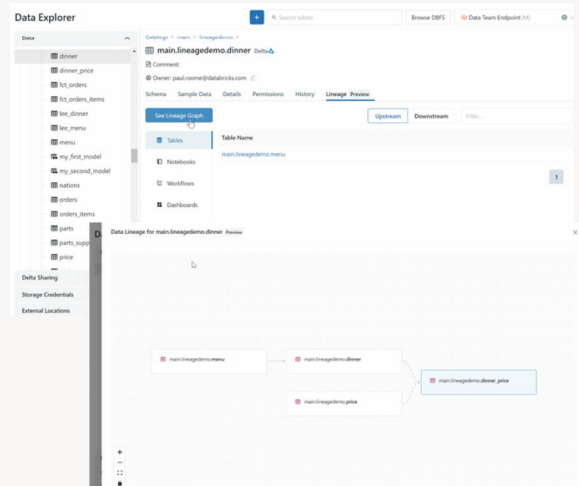
# Centralized governance for data and AI

- Create a single source of truth for your data with centralized metadata and user management
- Centrally manage access permissions and audit controls for files, tables across all workspaces and workloads using open standard ANSI SQL
- Enable fine-grained access controls on tables, files, rows, and columns



# Automated lineage for all workloads

- Get an end-to-end visibility into how data flows in your organization from source to consumption with built-in data lineage
- View lineage across tables, columns, notebooks, workflows, dashboards
- Captured in real time across all workloads in SQL, Python, Scala, and R
- Export API to enable partner integrations



<https://learn.microsoft.com/en-us/azure/databricks/data-governance/unity-catalog/data-lineage>

Unity Catalog captures runtime data lineage across queries run on Azure Databricks. Lineage is supported for all languages and is captured down to the column level. Lineage data includes notebooks, jobs, and dashboards related to the query. Lineage can be visualized in Catalog Explorer in near real time and retrieved programmatically using the lineage system tables.

Interactive Demo

<https://app.getreprise.com/launch/MnqjQDX/>

Unity Catalog's lineage tracking provides:

- **End-to-end transparency:** Shows exactly how data flows from raw sources to curated tables to BI dashboards.
- **Improved data quality:** You can trace errors or quality issues back to their origins.
- **Impact analysis:** Before modifying a dataset, you can see what downstream tables, reports, or ML pipelines will be affected.
- **Governance & compliance:** Unity Catalog enforces permissions and masks details if the user lacks access, ensuring secure visibility.

Unity Catalog automatically tracks lineage across:

**1. All languages & workloads:**

- Python, SQL, Scala, R, and both batch + streaming.

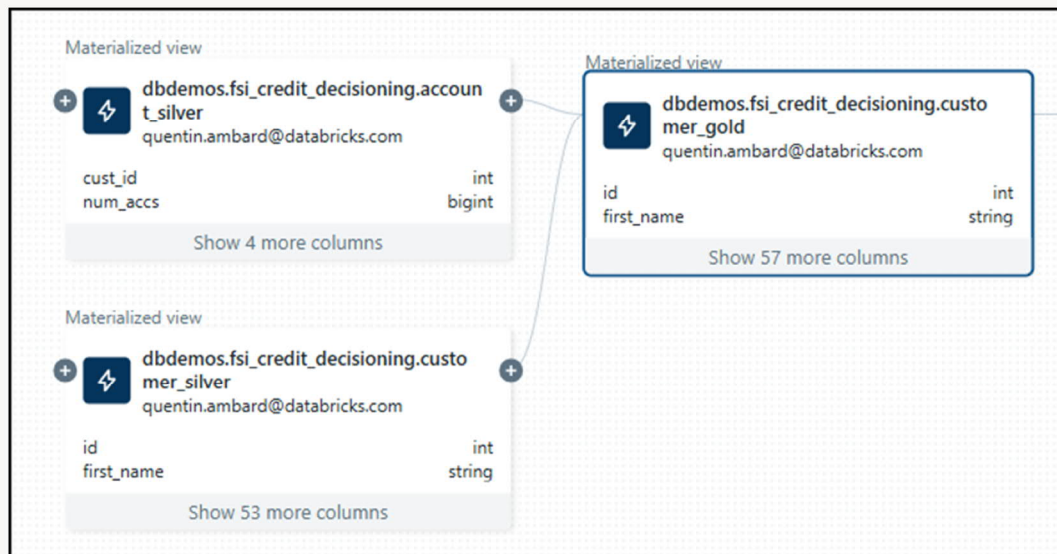
**2. Column-level lineage**

- Shows exactly which columns contribute to transformations

**3. Notebook, job, and dashboard usage**

- Helps understand how data gets consumed.

# Databricks Lineage – Tables



12

## 1. Unity Catalog lineage is generated by the *Databricks runtime*, not Delta Lake

Unity Catalog captures **runtime data lineage** whenever queries run on Databricks—across **SQL, Python, Scala, and R**—and across **all table types**, as long as those tables are **registered in Unity Catalog**.

This capture is done by the **UC lineage engine**, *not* by Delta Lake.

Unity Catalog captures lineage by observing the Spark logical plan during execution. For this to happen, UC must recognize the objects being queried.

## 2. Delta Lake is *not* required for lineage—but some lineage features depend on Delta

Unity Catalog can track lineage for any table format registered in UC.

However, **streaming lineage requires Delta Lake tables**:

*“Lineage tracking of **streaming between Delta tables** requires Databricks Runtime 11.3 LTS or above.”*

This means lineage is UC-based, but Delta Lake is needed for streaming workflows.

## Lineage System Table in Unity Catalog - YouTube

<https://youtu.be/bZXswjZ0avA>

# Databricks Lineage - Columns

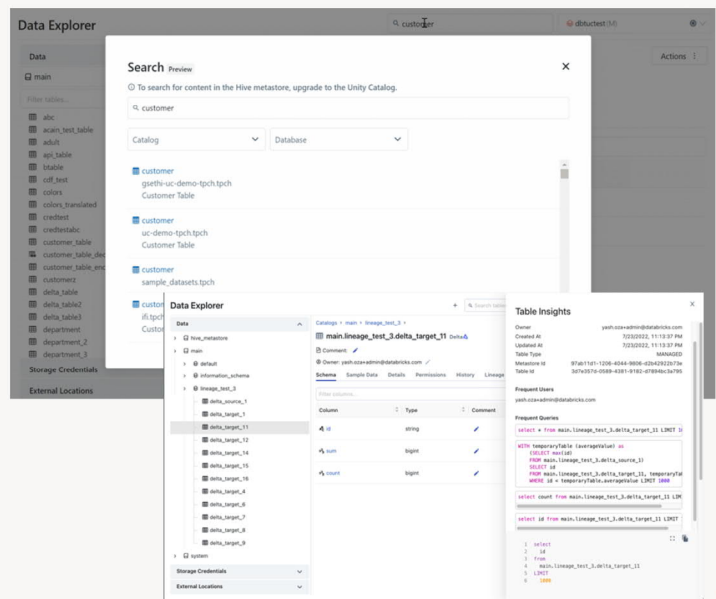
| Columns    |        |                                                                                   |
|------------|--------|-----------------------------------------------------------------------------------|
| Lineage    |        |                                                                                   |
| Column     | Type   | Lineage                                                                           |
| id         | int    |  |
| first_name | string |  |
| last_name  | string |  |
| email      |        |                                                                                   |
| gender     |        |                                                                                   |

| Materialized view                                                                                                                                                   |        | Table                                                                                                                                                         |        |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
|  dbdemos.fsi_credit_decisioning.custo<br>mer_gold<br>quentin.ambard@databricks.com |        |  dbdemos.fsi_credit_decisioning.custo<br>mer_gold_features<br>account users |        |
| id                                                                                                                                                                  | int    | id                                                                                                                                                            | int    |
| first_name                                                                                                                                                          | string | first_name                                                                                                                                                    | string |
| last_name                                                                                                                                                           | string | last_name                                                                                                                                                     | string |
| email                                                                                                                                                               | string | email                                                                                                                                                         | string |
| gender                                                                                                                                                              | string | gender                                                                                                                                                        | string |



# Built-in search and discovery

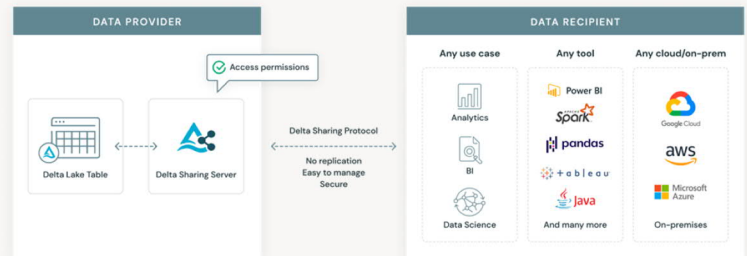
- Quickly find, understand, and reference data from across your data estate with a unified data browsing experience
- Secure by default—leverages common permission model from Unity Catalog
- Auto-generated data insights to understand your data





# Secure data sharing and collaboration

- Securely share live data from your lakehouse to any computing platform
- Eliminate unnecessary data replication, and vendor lock-in
- Centrally manage, govern, monitor usage and audit access to shared data with ease

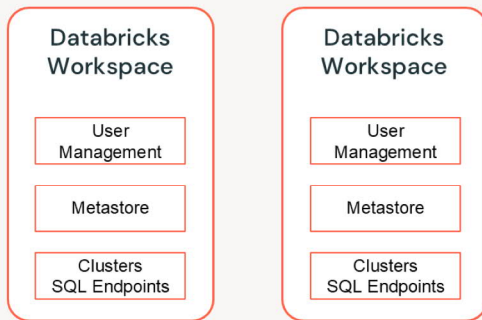


**Delta Sharing** is an **open protocol**, not a proprietary feature. It decouples the data provider from the data consumer, allowing for secure, live data exchange regardless of whether the recipient uses Databricks.

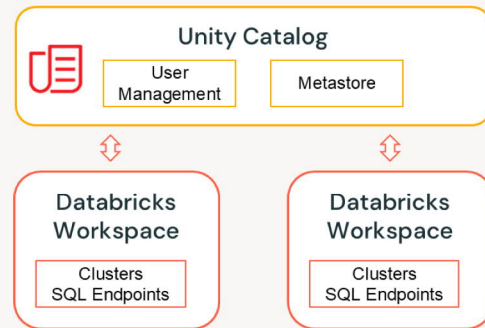
# Introducing Unity Catalog

Unified view of your data estate via a centralized metadata and user mgmt

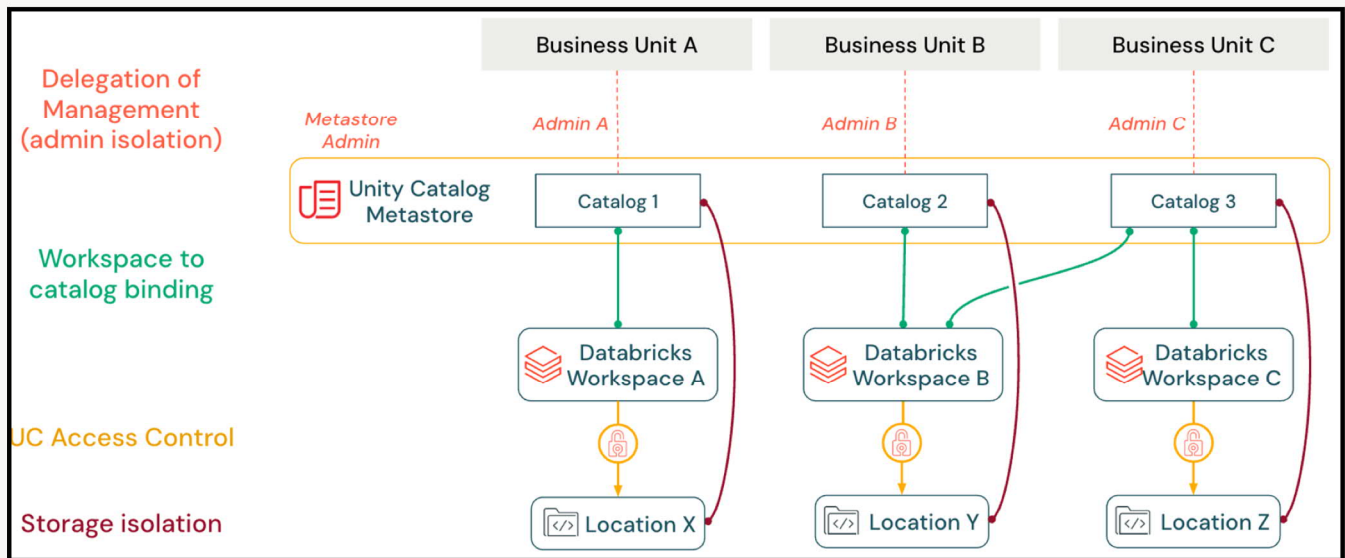
## Without Unity Catalog



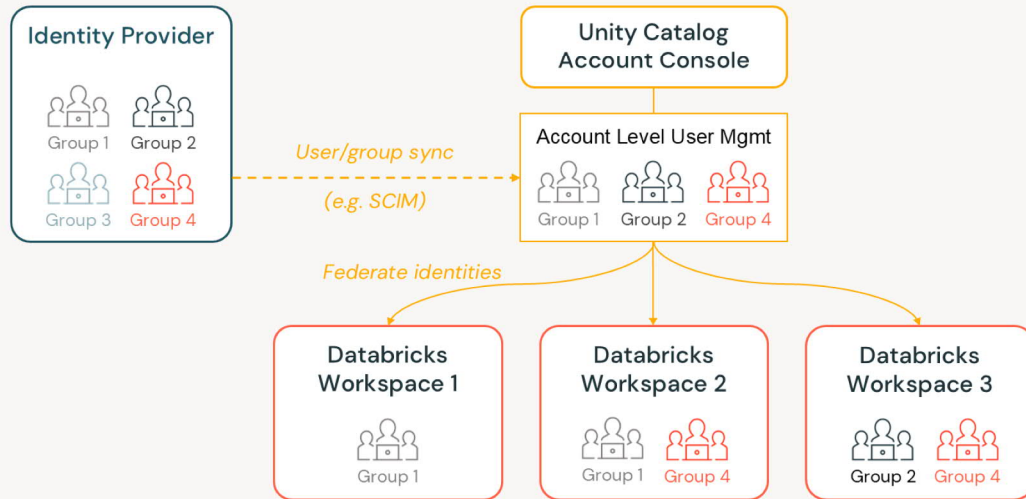
## With Unity Catalog



# Introducing Unity Catalog

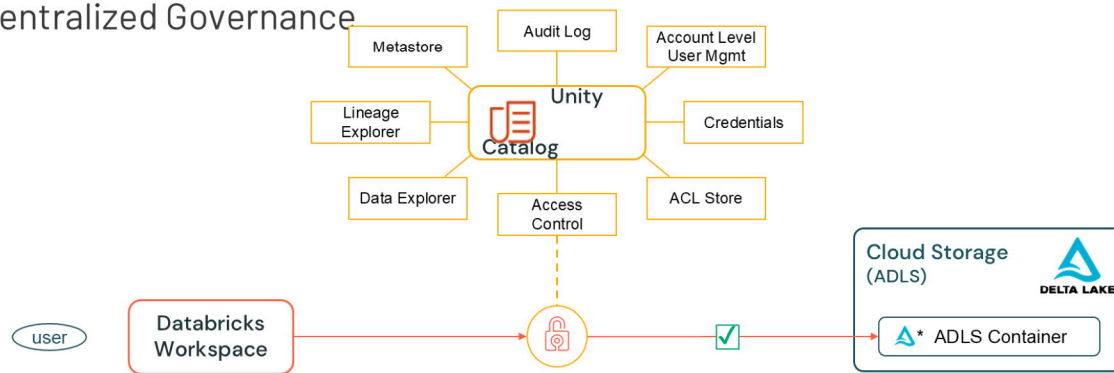


# Identity Federation with Unity Catalog



# Unity Catalog – Components

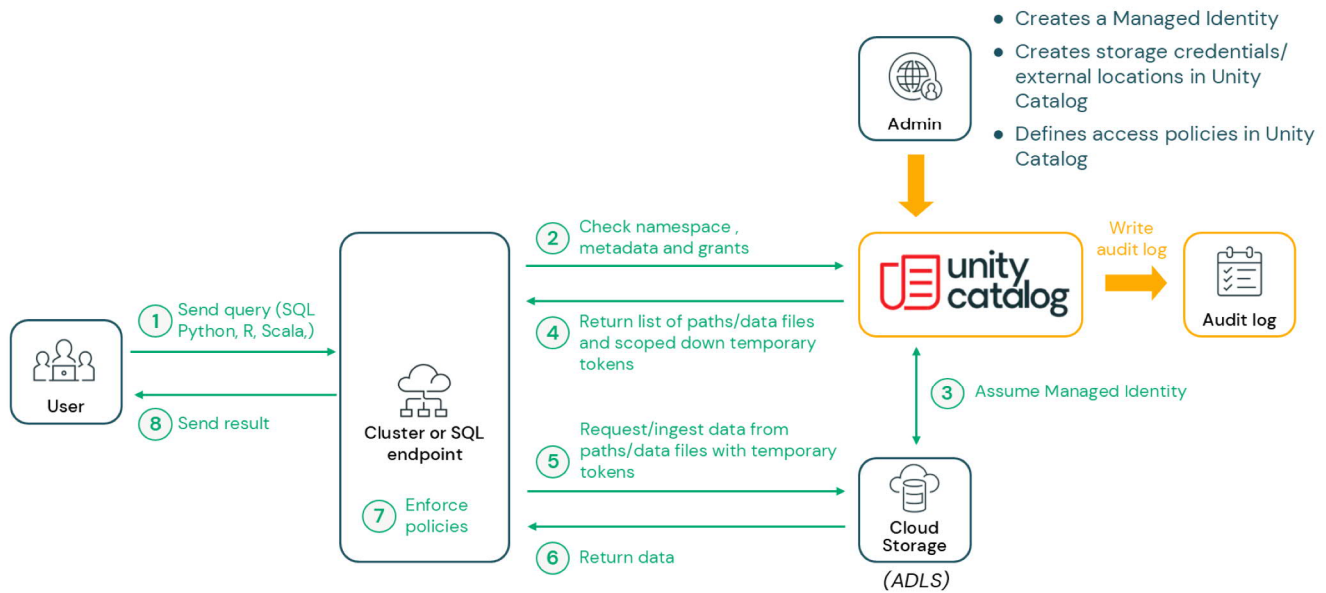
## Centralized Governance



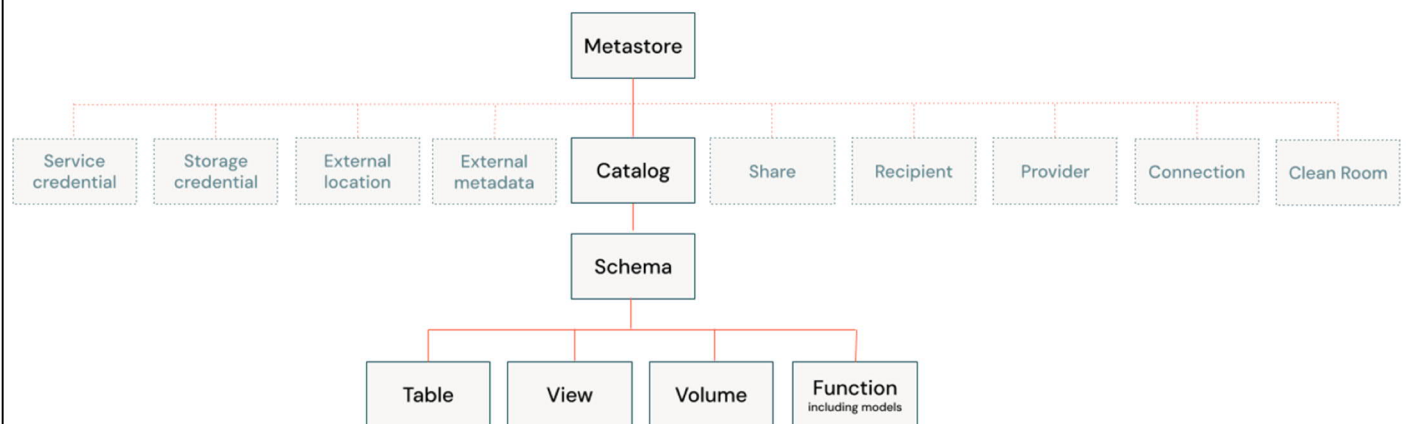
\* Unity Catalog will support any data format (table or raw files)



# Life of a query with Unity Catalog



# Unity Catalog Objects



<https://learn.microsoft.com/en-us/azure/databricks/data-governance/unity-catalog/>

In a Unity Catalog metastore, the three-level database object hierarchy consists of catalogs that contain schemas, which in turn contain data and AI objects, like tables and models. This hierarchy is represented as a three-level namespace (catalog.schema.table-etc) when you reference tables, views, volumes, models, and functions.

## Level one:

- **Catalogs** are used to organize your data assets and are typically used as the top level in your data isolation scheme. Catalogs often mirror organizational units or software development lifecycle scopes. See [What are catalogs in Azure Databricks?](#).
- **Non-data securable objects**, such as storage credentials and external locations, are used to manage your data governance model in Unity Catalog. These also live directly under the metastore. They are described in more detail in [Securable objects that Unity Catalog uses to manage access to external data sources](#).

## Level two:

- **Schemas** (also known as databases) contain tables, views, volumes, AI models, and functions. Schemas organize data and AI assets into logical categories that are more granular than catalogs. Typically a schema represents a single use case, project, or team sandbox.

## Level three:

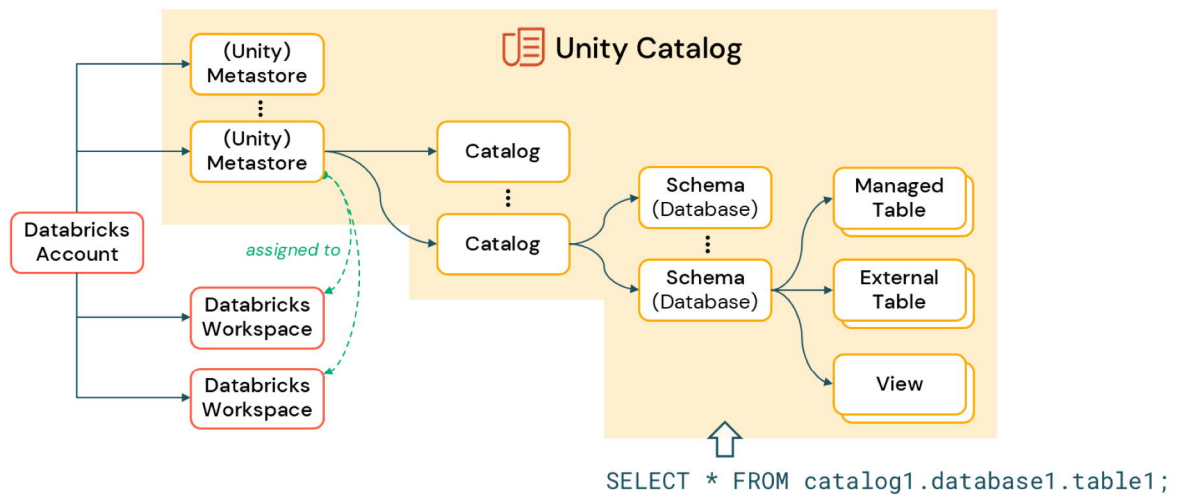
- **Tables** are collections of data organized by rows and columns. Tables can be either *managed*, with Unity Catalog managing the full lifecycle of the table, or *external*,

with Unity Catalog managing access to the data from within Azure Databricks, but not managing access to the data in cloud storage from other clients. See [Azure Databricks tables](#) and [Managed versus external tables and volumes](#).

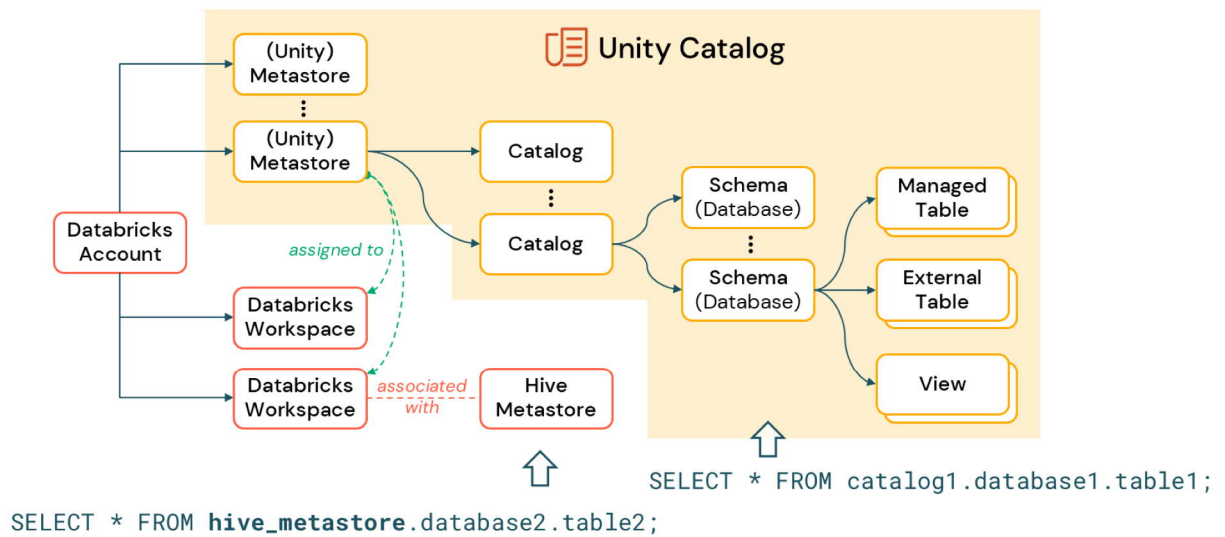
- **Views** are saved queries against one or more tables. See [What is a view?](#).
- **Volumes** represent logical volumes of data in cloud object storage. You can use volumes to store, organize, and access files in any format, including structured, semi-structured, and unstructured data. Typically they are used for non-tabular data. Volumes can be either *managed*, with Unity Catalog managing the full lifecycle and layout of the data in storage, or *external*, with Unity Catalog managing access to the data from within Azure Databricks, but not managing access to the data in cloud storage from other clients. See [What are Unity Catalog volumes?](#) and [Managed versus external tables and volumes](#).
- **Functions** are units of saved logic that return a scalar value or set of rows. See [User-defined functions \(UDFs\) in Unity Catalog](#).
- **Models** are AI models packaged with MLflow and registered in Unity Catalog as functions. See [Manage model lifecycle in Unity Catalog](#).



# The three level namespace of UC



# Hive Metastore is integrated into UC



# Tables (Managed vs External Tables)

- **Managed Tables (The "Default" Choice)**
- Unity Catalog manages both the **metadata** and the **underlying data files**.
- Data is stored in the "Managed Storage" location defined at the Metastore, Catalog, or Schema level (usually a dedicated S3/Azure/ADLS bucket).
- **Lifecycle:** If you run DROP TABLE, Unity Catalog deletes **both** the metadata and the physical data files from storage within 30 days (permanent deletion).



# External Tables

- Tables where Unity Catalog manages only the **metadata**, while the data files live in a location you provide.
- **Storage Location:** You must specify/register a AWS/ADLS/GCP Storage LOCATION when creating the table.
  - ADLS: Azure Data Lake Storage
- **Lifecycle:** If you run DROP TABLE, Unity Catalog **only** deletes the metadata (the pointer). Your data files remain safe in your S3/ADLS bucket.



## Step 1: Create a Storage Credential

- In the Catalog Explorer, create a **Storage Credential**.
- **Under the Hood:** This uses a **Cloud Managed Identity** (Azure) or an **IAM Role** (AWS) that has Read/Write permissions on your specific bucket.

## Step 2: Define the External Location

- Create the **External Location** object by linking a specific S3/ADLS path (e.g., s3://my-data-bucket/gold/) to the **Storage Credential** created in Step 1.
- You can have multiple External Locations using the same Storage Credential.

# Unity Catalog Volumes

- Volumes provide capabilities for accessing, storing, governing, and organizing files.
- While tables govern tabular data, volumes govern non-tabular data of any format, including structured, semi-structured, or unstructured.
- Volumes are available in two types:
  - **Managed volumes:** For simple Databricks-managed storage.
  - **External volumes:** For adding governance to existing cloud object storage locations.



<https://learn.microsoft.com/en-us/azure/databricks/volumes/>

- Volumes are Unity Catalog objects that enable governance over non-tabular datasets. Volumes represent a logical volume of storage in a cloud object storage location.
- Volumes provide capabilities for accessing, storing, governing, and organizing files while tables govern tabular data, volumes govern non-tabular data of any format, including structured, semi-structured, or unstructured.
- Databricks recommends using volumes to govern access to all non-tabular data.

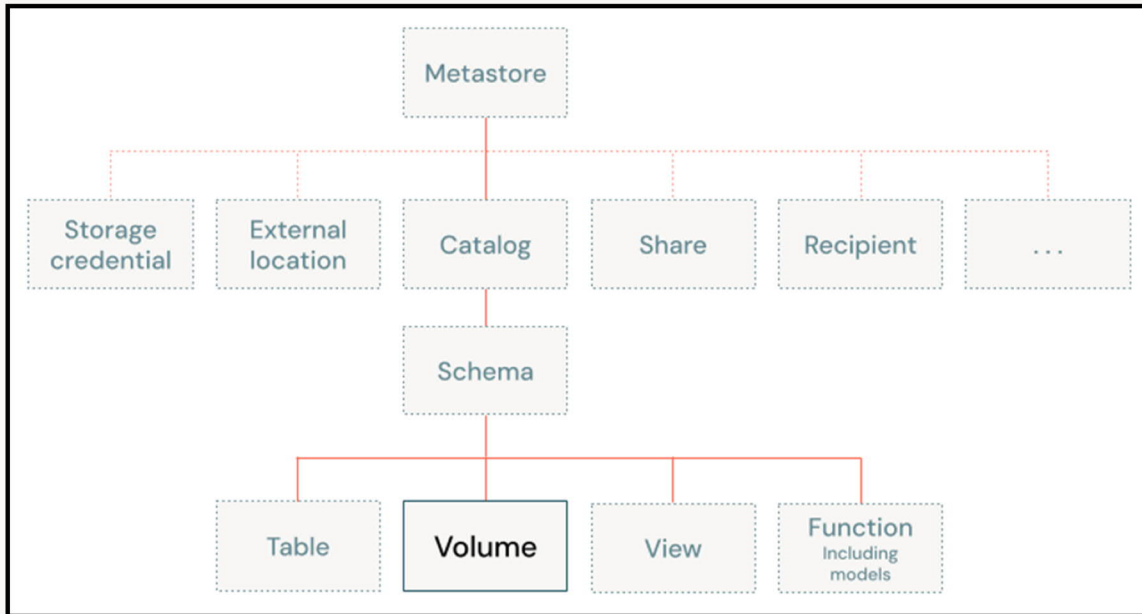
Volumes are available in two types:

- **Managed volumes:** For simple Databricks-managed storage.
- **External volumes:** For adding governance to existing cloud object storage locations.

## Unity Catalog Volumes (cont...)

| Feature                  | Managed volumes                                            | External volumes                                          |
|--------------------------|------------------------------------------------------------|-----------------------------------------------------------|
| <b>Storage location</b>  | Created inside the UC-managed storage for the schema       | Registered against an existing cloud object storage path  |
| <b>Data lifecycle</b>    | UC manages layout and deletion (7-day retention on delete) | Data remains in cloud storage when you drop the volume    |
| <b>Access control</b>    | All access goes through UC                                 | UC governs access, but external tools can use direct URIs |
| <b>Migration needed?</b> | No                                                         | No—use existing storage paths as-is                       |
| <b>Typical use case</b>  | Simplest option for Databricks-only workloads              | Mixed Databricks and external system access               |

## Unity Catalog Volumes (cont...)



Volumes sit at the third level of the Unity Catalog three-level namespace (catalog.schema.volume):

- The path to access volumes is the same whether you use Apache Spark, SQL, Python, or other languages and libraries. This differs from legacy access patterns for files in object storage bound to an Azure Databricks workspace.

The path to access files in volumes uses the following format:

- `/Volumes/<catalog>/<schema>/<volume>/<path>/<file-name>`
- Azure Databricks also supports an optional `dbfs:/` scheme when working with Apache Spark, so the following path also works:
- `dbfs:/Volumes/<catalog>/<schema>/<volume>/<path>/<file-name>`

# Creating Volumes

- To create a managed volume, use the following syntax:
  - `CREATE VOLUME <catalog>.<schema>.<volume-name>;`
- To create an external volume, add the `LOCATION` clause, as in the following example:

```
CREATE EXTERNAL VOLUME
```

```
    <catalog>.<schema>.<external-volume-name>
```

```
LOCATION 'abfss://<container-name>@<storage-account>  
        .dfs.core.windows.net/<path>/<directory>';
```

- Drop Volume

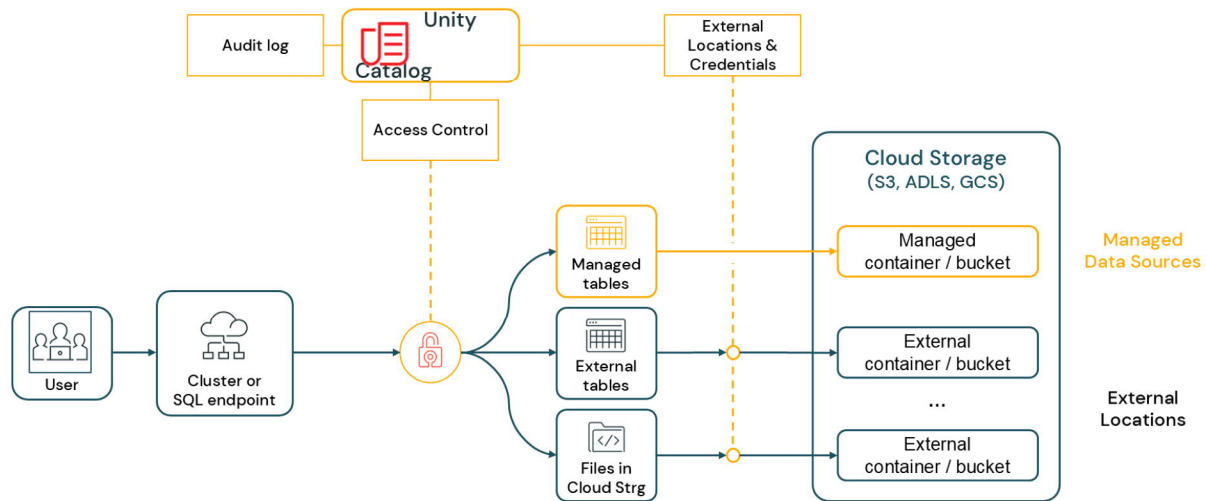
```
DROP VOLUME IF EXISTS <volume-name>;
```



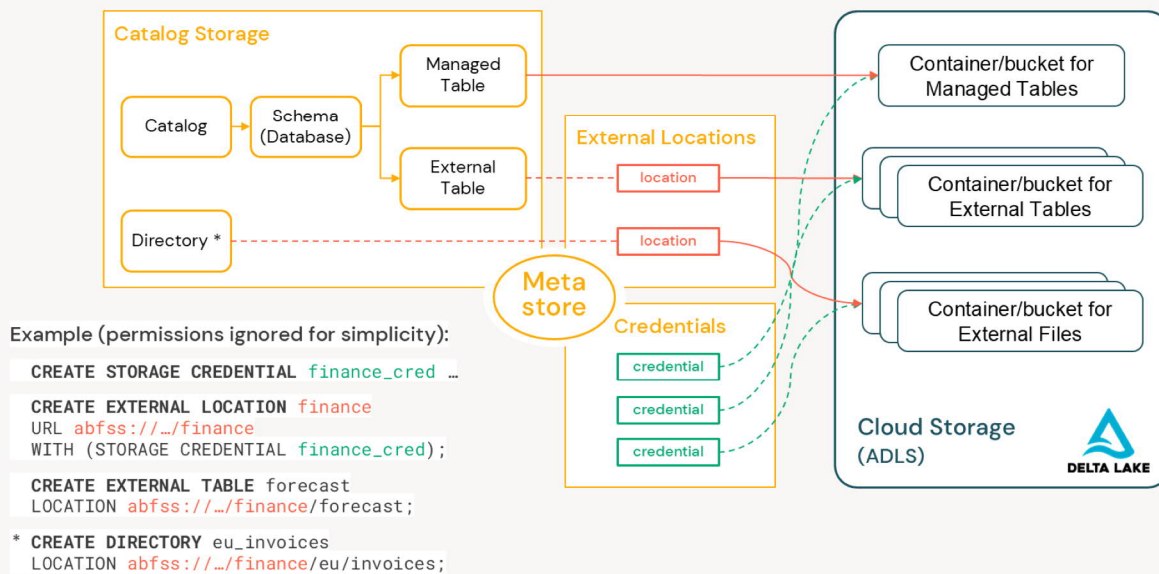
- <https://learn.microsoft.com/en-us/azure/databricks/volumes/utility-commands>



# Managed Data Sources & External Locations



# Metastore, external locations and credentials



\* coming soon

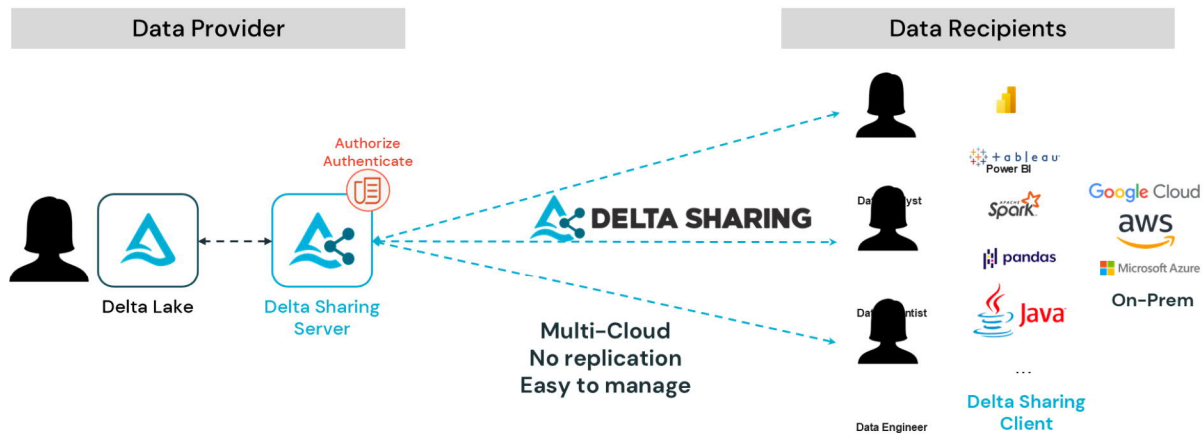


# Unity Catalog and Delta Sharing



# A simple, open and easy approach to data sharing

Reduce data sharing and collaboration from days to real-time



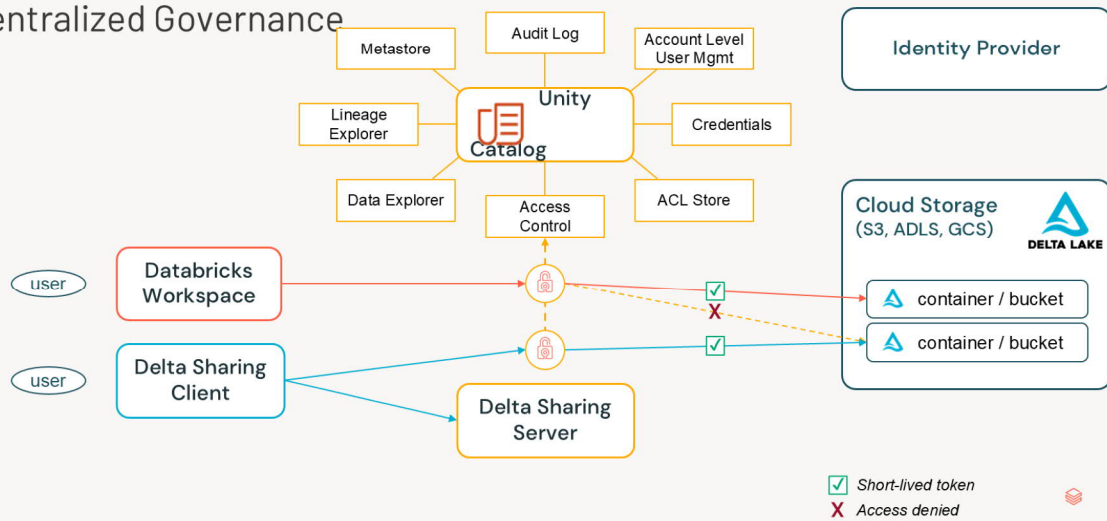
33

**Databricks-to-Databricks Sharing:** If both the provider and recipient are on Databricks (with Unity Catalog enabled), they share data using a **Sharing Identifier** (a unique metastore ID). No credential files or tokens are needed; the data appears directly in the recipient's Catalog Explorer.

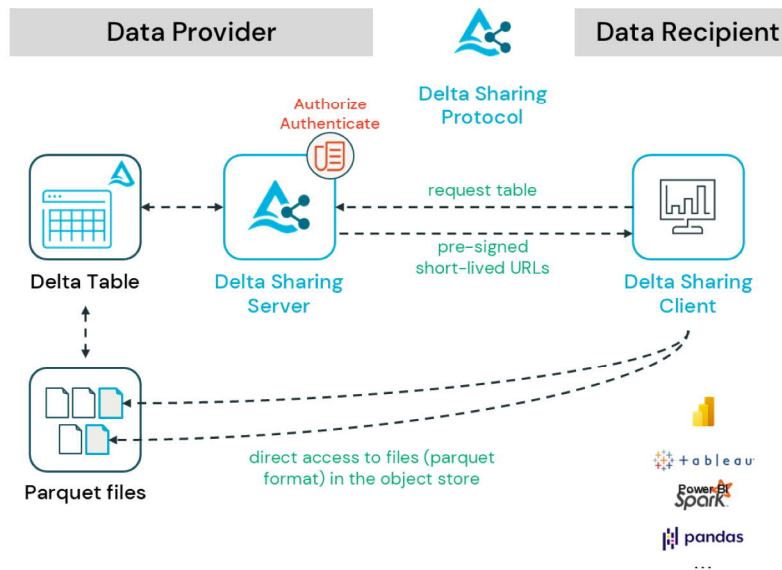
**Open Sharing:** The provider creates a **Recipient object** in Databricks and generates an activation link. The recipient (who could be using Power BI, a local Python script, or even a different cloud provider) uses that link to download a **credential file** (JSON). They then use open-source Delta Sharing connectors to query the data.

# Unity Catalog & Delta Sharing - Components

## Centralized Governance



# Under the Hood



## Delta Sharing Protocol:

- Client authenticates to Sharing Server
- Client requests a table (including filters)
- Server checks access permissions
- Server generates and returns pre-signed short-lived URLs
- Client uses URLs to directly read files from object storage

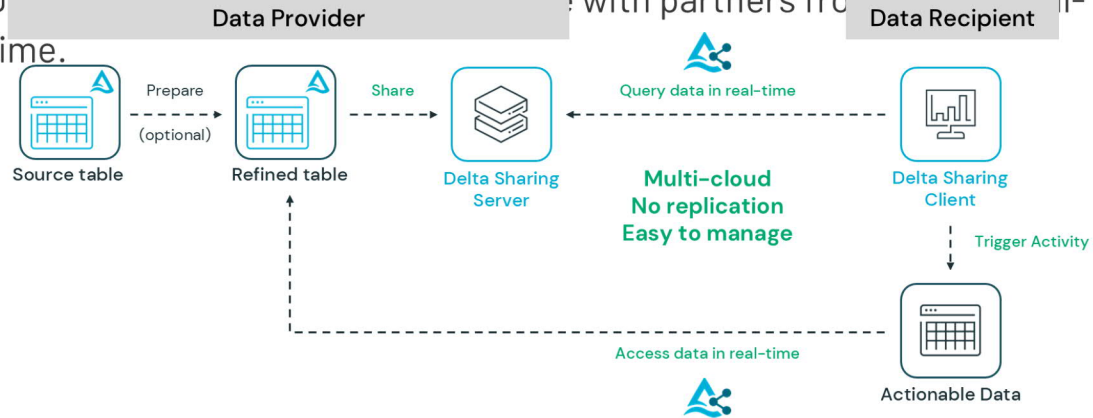
## Notes:

- Sharing happens on Delta part files, supporting full tables, partitions, delta versions, ...
- Client is system independent, just needs to be able to read parquet files
- In Databricks Sharing Server and ACL checks are integrated with Unity Catalog

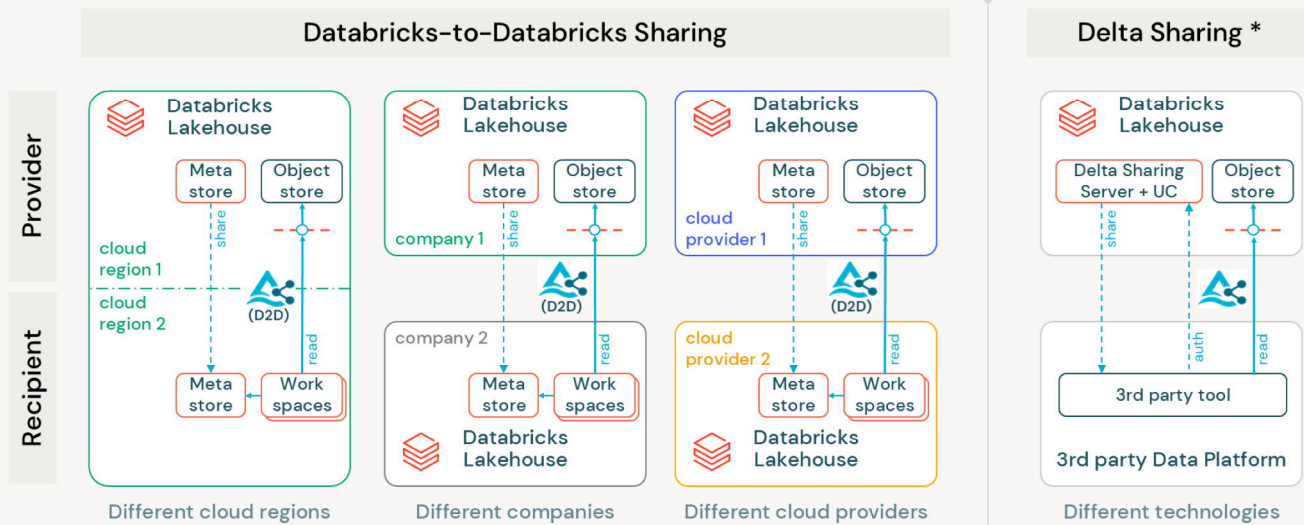


## Streamlined Sharing with Delta Sharing

Delta Sharing auto-collaboration time with partners from dev to real-time.



# Databricks-to-Databricks (D2D) Sharing



Provide access to the data on a network level

\* Implementing the Delta Sharing protocol, 3rd parties can act as a data provider

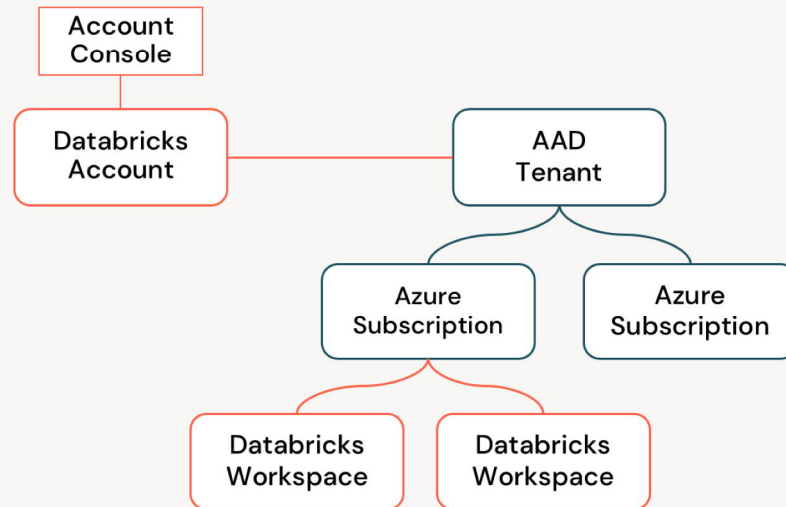




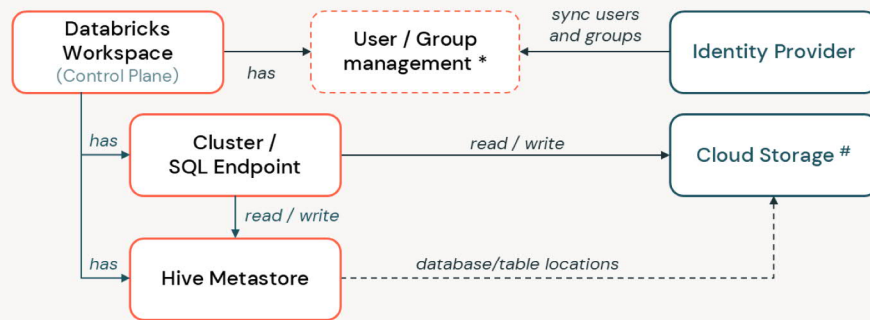
# Appendix



# Databricks Accounts and the Cloud Provider Hierarchy



# Object relations before Unity Catalog



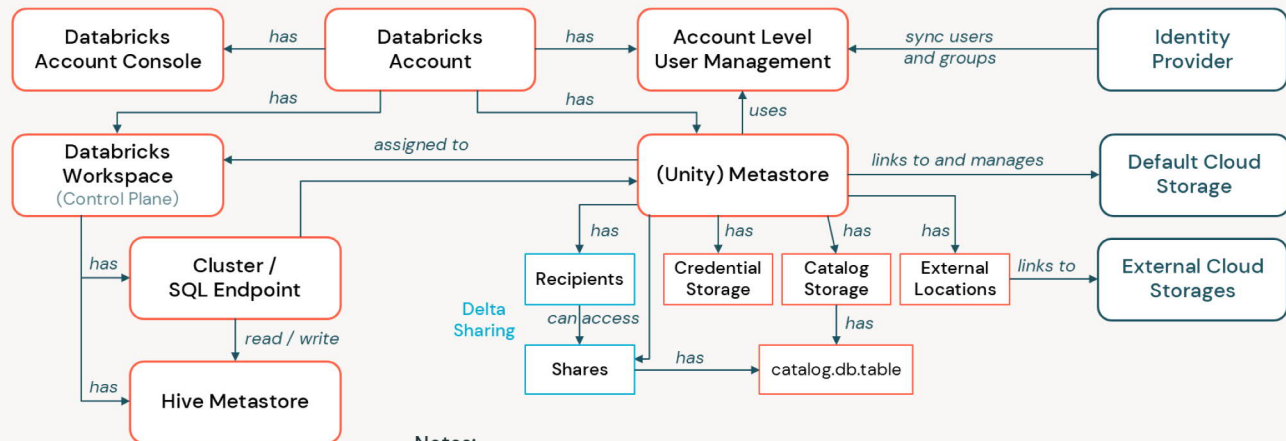
\* An integrated module of every Databricks Workspace

\* Could also be an external Metastore

# Any accessible cloud storage or the root container (dbfs)



# Object relations with Unity Catalog (UC)



## Notes:

- Using Unity Catalog is optional. Workspaces can still use a Hive Metastore only
- There can be more than one Unity Metastore (UC) per Databricks Account (e.g. for regional isolation or for isolation of lines of business)
- Every workspace can only attach to one UC Metastore, however one Unity Metastore can be assigned to several Workspaces

